Sentra: Cron Job Demo (Simple Guide)

Every command explained in plain words: what to type, what each part means, what you will see, and what to say to the teacher.

1. The idea in one minute

A **cron job** is a program that runs on its own on a schedule, for example "back up the database every night". The problem: if it stops running, **nobody notices**. There is no error and no website that goes down. It just silently doesn't happen.

Sentra's solution: the job sends Sentra a short message, called a **ping**, every time it finishes. Sentra knows how often the pings should arrive. If a ping is late, or the job says "I failed", Sentra marks the job **DOWN** and opens an **incident**. This idea is called a **dead man's switch**: silence itself is the alarm.

```
Windows Task Scheduler  (runs every 1 minute)
   └─ runs demo-cron-job.ps1  (zips the docs folder = a pretend "backup")
        ├─ at the start   ─►  /ping/<token>/start
        ├─ if it worked   ─►  /ping/<token>        ─►  Sentra: HEALTHY (green)
        └─ if it failed   ─►  /ping/<token>/fail   ─►  Sentra: DOWN (red) + incident

   no ping at all for (1 min + 1 min allowed delay)  ─►  Sentra: DOWN + incident
```

Words you will use

Word	Meaning
Ping	The job visiting a special Sentra web address to say "I ran". Just a normal web request.
Token	A long random code in that address. It tells Sentra *which* job is pinging. It works like a password.
Task Scheduler	Windows' built-in tool for running programs on a schedule. It is the Windows version of Linux `cron`.
Allowed delay	Extra waiting time before Sentra raises the alarm, so a ping that is a few seconds late isn't treated as a failure.
PENDING	The check was just created and no ping has arrived yet. Sentra waits and does not alarm.
HEALTHY	The last ping came on time and said "success".
DOWN	The ping is late, or the job said "I failed". An incident is open.
Incident	One record of an outage: when it started, when it ended, and why.
Push model	The job contacts Sentra. Sentra never needs to log in to the job's computer.

The three things running during the demo

Part	What it is	Where
Database	PostgreSQL in Docker. Stores checks, pings and incidents.	port 55432
Backend	The Sentra server (Java / Spring Boot). Receives pings and decides HEALTHY or DOWN.	port 8081
Dashboard	The website you show the teacher (React).	port 5173

2. Part A: Start Sentra (before the teacher arrives)

Open **three PowerShell windows** and keep all of them open during the demo.

A1. Start the database (any window)

First open **Docker Desktop** and wait until it says it is running.

```
cd C:\Users\shubh\OneDrive\Documents\Sentra
docker compose up -d
```

EACH PART

`cd ...\Sentra`	Go to the project folder ("change directory").
`docker compose up`	Read `docker-compose.yml` and start what it describes: the PostgreSQL database.
`-d`	"Detached": run in the background so the window is free again.

You will see: `Container sentra-db Started` (or `Running` if it was already on).

A2. Start the backend (window 1)

```
cd C:\Users\shubh\OneDrive\Documents\Sentra\backend
mvn spring-boot:run "-Dspring-boot.run.profiles=dev"
```

EACH PART

`mvn`	Maven, the Java build tool. It compiles and runs the project.
`spring-boot:run`	"Start the Spring Boot server."
`profiles=dev`	Use development settings. This gives the server a test secret key for logins, so nothing else needs setting up.

You will see: after about 15 seconds, `Tomcat started on port 8081` and `Started SentraApplication`. Leave this window open: closing it stops Sentra.

If `mvn` is "not recognised": close and reopen the terminal, or run this in the same window first. It reloads Windows' list of program folders:

```
$machine = [Environment]::GetEnvironmentVariable("Path","Machine")
$user    = [Environment]::GetEnvironmentVariable("Path","User")
$env:Path = "$machine;$user"
```

A3. Start the dashboard (window 2)

```
cd C:\Users\shubh\OneDrive\Documents\Sentra\frontend
npm run dev
```

EACH PART

`npm run dev`	Start the website in development mode. It also forwards the website's requests to the backend on port 8081.

You will see: `Local: http://localhost:5173/` . Open it in the browser and log in.

A4. Check the backend is alive (window 3: use this one for everything below)

```
curl.exe http://localhost:8081/api/v1/ping
```

EACH PART

`curl.exe`	A tool that visits a web address from the terminal. Write `.exe` : in PowerShell, plain `curl` is a different command.
`/api/v1/ping`	A health address in the backend that just answers "ok".

You will see: `{"status":"ok"}`

3. Part B: Create the demo job

Step 1. Create the check in the dashboard (browser)

1. Click **+ Add check**, then choose the ⏱ **Cron job** type.
2. Name: `Demo backup job`
3. Job runs every: **1 minute**. Allowed delay (minutes): **1**
4. Click **Add check**.

In plain words: you tell Sentra "this job will ping me every minute; if 2 minutes pass with no ping, raise the alarm."

You will see: state **PENDING**, "Last ping: none yet", and a **Ping URL** like `http://localhost:5173/ping/AbC123xyz...` . The long text after `/ping/` is the **token**. Copy it.

"Sentra created a random, unguessable token. Knowing this URL is the job's only password."

Step 2. Save the token

```
$token = "PASTE-YOUR-TOKEN-HERE"
```

In plain words: store the token in a PowerShell **variable** named `$token` , so you don't have to paste the long text again. Variables start with `$` . They are forgotten when you close the window.

You will see: nothing. No output means it worked.

Step 3. Send one ping by hand

```
curl.exe http://localhost:8081/ping/$token
```

In plain words: you pretend to be the job and say "I ran successfully". PowerShell replaces `$token` with your real token. This proves the token is correct *before* you automate anything.

You will see: `OK` . In the browser, "Last ping" says **just now** and within about 5 seconds the state turns **HEALTHY**.
`404` means the token was copied wrongly: redo Step 2.

Step 4. Create the scheduled task that runs every minute (the long one, read slowly)

This is three small commands. The first two only **prepare text**; the third one actually creates the task.

4A. REMEMBER WHERE THE SCRIPT IS

```
$script = "C:\Users\shubh\OneDrive\Documents\Sentra\scripts\demo-cron-job.ps1"
```

Saves the script's file path in the variable `$script`.

4B. BUILD THE COMMAND THAT WINDOWS WILL RUN EVERY MINUTE

```
$run = "conhost.exe --headless powershell.exe -NoProfile -ExecutionPolicy Bypass -File $script -Token $token"
```

`conhost.exe --headless`	Run it **without opening a window**, so no black box pops up every minute.
`powershell.exe`	Start PowerShell, which will run our script.
`-NoProfile`	Skip personal PowerShell startup settings. Faster, and always behaves the same.
`-ExecutionPolicy Bypass`	Windows blocks `.ps1` scripts by default. This allows our script to run.
`-File $script`	Which script to run.
`-Token $token`	Hand our token to the script, so it pings the right check.

4C. CREATE THE TASK

```
schtasks /Create /TN "SentraDemoJob" /SC MINUTE /MO 1 /TR $run /F
```

`schtasks`	The command-line remote control for Windows Task Scheduler.
`/Create`	Make a new task.
`/TN "SentraDemoJob"`	**T**ask **N**ame. All later commands use this name.
`/SC MINUTE`	**SC**hedule: repeat by minutes.
`/MO 1`	**MO**difier: every **1** minute. (`/SC HOURLY /MO 6` would mean every 6 hours.)
`/TR $run`	**T**ask **R**un: the command to run, the one we built in 4b.
`/F`	**F**orce: if a task with this name already exists, replace it.

You will see: `SUCCESS: The scheduled task "SentraDemoJob" has successfully been created.`

"*Task Scheduler is Windows' version of cron. Every minute it runs our script, which zips the docs folder as a pretend backup and then pings Sentra.*"

Step 5. Let the task run on battery

```
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries
Set-ScheduledTask -TaskName "SentraDemoJob" -Settings $settings
```

`New-ScheduledTaskSettingsSet`	Prepare a set of task settings and save them in `$settings`.
`-AllowStartIfOnBatteries`	Start the task even when the laptop is unplugged.
`-DontStopIfGoingOnBatteries`	Don't stop it if the charger is pulled out while it runs.
`Set-ScheduledTask ...`	Apply those settings to our task.

Why: by default Windows **silently skips** scheduled tasks on battery. The job would just never run.

You will see: a short description of the task and no red error text.

"*This is exactly the kind of silent failure Sentra catches: the job never ran, and nothing crashed.*"

Step 6. Run it once now and check

```
schtasks /Run /TN "SentraDemoJob"
Get-Content "$env:TEMP\sentra-demo-backups\job.log" -Tail 3
```

`schtasks /Run`	Run the task **right now**, instead of waiting for the next minute.
`Get-Content ... -Tail 3`	Show the last 3 lines of the job's log file. (`Get-Content` = show a file.)
`$env:TEMP`	Your Windows temp folder. The script keeps its zips and log there.

You will see: a log line like `2026-10-28 18:30:05Z OK docs-20261028-183004.zip`. The dashboard shows **HEALTHY**, and "Last ping" now updates every minute on its own.

4. Part C: The failure demos

What to point at on the dashboard

On screen	What it means
PENDING	No ping yet. Sentra doesn't alarm for a job that hasn't had a chance to run.
HEALTHY (green)	Last ping was on time and said "success".
DOWN (red)	Ping is late or the job reported a failure. An incident is open.
Last ping	When the job last reported. Shows "(reported a failure)" after a `/fail` ping.
Run tape / Recent runs	Sentra's verdict after each ping and at each deadline: green = fine, red = late or failed.
Incidents	One row per outage: start time, duration, reason.

Step 7. Demo 1: the job runs but fails

```
New-Item -ItemType File "$env:TEMP\sentra-demo-backups\FAIL"
schtasks /Run /TN "SentraDemoJob"
```

`New-Item -ItemType File ...\FAIL`	Create an empty file called `FAIL`. The script checks for this file; if it exists, the script fails on purpose.
`schtasks /Run`	Run the job now, so you don't wait for the next minute.

You will see: the log says `FAIL simulated failure ...`. Within seconds the check is **DOWN** with an open incident: "Job reported a failure".

"The FAIL file stands in for a real problem like a full disk. The job noticed it failed and told Sentra through the /fail address."

FIX IT

```
Remove-Item "$env:TEMP\sentra-demo-backups\FAIL"
schtasks /Run /TN "SentraDemoJob"
```

`Remove-Item` deletes the FAIL file, so the next run succeeds.

You will see: a new `OK` line. Within about 5 seconds the check is **HEALTHY** and the incident says **Resolved**, with how long it lasted.

Step 8. Demo 2: the job stops running completely

```
schtasks /Change /TN "SentraDemoJob" /DISABLE
```

`/Change ... /DISABLE`	Switch the task's schedule off. The job simply stops running and no pings are sent.

In plain words: this copies a crashed server or a deleted cron entry. **No error appears anywhere.** The only sign is that the pings stop.

You will see: "Last ping" stops changing. About **2 minutes** after the last ping (1 min period + 1 min allowed delay), the check turns **DOWN**: "No ping for ..., expected every 1 min".

"You can't detect something missing by waiting for an event. Sentra expects the ping, so the silence itself becomes the alarm."

FIX IT

```
schtasks /Change /TN "SentraDemoJob" /ENABLE
schtasks /Run /TN "SentraDemoJob"
```

`/ENABLE` turns the schedule back on, and `/Run` sends a ping immediately instead of waiting for the next minute.

You will see: within about 5 seconds the check is **HEALTHY** again and the incident is **Resolved**.

Handy commands to show during the demo

`Get-Content "$env:TEMP\sentra-demo-backups\job.log" -Tail 5`	What the job did in its last 5 runs.
`schtasks /Query /TN "SentraDemoJob" /V /FO LIST`	The task's details: last run time, result, next run. (`/V` = verbose, `/FO LIST` = one item per line.)
`Get-ChildItem "$env:TEMP\sentra-demo-backups"`	List the backup zip files the job made. (`Get-ChildItem` = list a folder.)

5. Part D: Clean up after the demo

Step 9. Delete the task and the check

```
schtasks /Delete /TN "SentraDemoJob" /F
```

Deletes the scheduled task. `/F` = don't ask "are you sure?".

You will see: `SUCCESS: The scheduled task "SentraDemoJob" was successfully deleted.`

Then, in the dashboard, open the check and scroll to **Manage check**:

- ⏸ **Pause** stops monitoring but keeps the history (▶ **Resume** starts it again), or
- **Delete check** → **Yes, delete** removes the check and all its history.

If you skip this, the check goes DOWN after about 2 minutes, because the pings stopped. That is correct behaviour.

OPTIONAL: REMOVE THE PRETEND BACKUPS AND LOG

```
Remove-Item "$env:TEMP\sentra-demo-backups" -Recurse -Force
```

`-Recurse` = also delete everything inside the folder. `-Force` = don't ask.

6. If something goes wrong

Problem	Why, and what to do
`mvn` not recognised	The terminal was opened before Maven was installed. Reopen it, or run the PATH line in A2.
"Port 8081 was already in use"	A backend is already running in another window. Use that one, or close it first.
Backend can't connect to database	Docker Desktop isn't running or the container stopped. Redo A1.
Login says "Something went wrong"	The backend isn't running, or it's on the wrong port. Check window 1 says `port 8081`.
`curl` gives 404	The token was copied wrongly. Copy it again from the check page and redo Step 2.
`curl` gives 403 / "Whitelabel Error Page"	The address is missing `/ping/<token>`. Only that path is open without logging in.
Check is DOWN but the task "should" be running	The laptop is on battery and Step 5 was skipped, or the task is disabled. Check with `schtasks /Query ...`. "Last Run Time 1999" means it never ran.
Task runs but no ping arrives	If the token starts with `-`, PowerShell mistakes it for an option. Create a new check to get a different token.
"No ping" in a new PowerShell window	`$token` was forgotten when the old window closed. Redo Step 2 in the new window.

7. One-page cheat sheet

Before the demo

Do this	Type
Start database	`docker compose up -d`
Start backend (window 1)	`cd backend` then `mvn spring-boot:run "-Dspring-boot.run.profiles=dev"`
Start dashboard (window 2)	`cd frontend` then `npm run dev`
Is the backend alive?	`curl.exe http://localhost:8081/api/v1/ping`

Set up the job (window 3)

Do this	Type
Save token	`$token = "..."`
Test ping by hand	`curl.exe http://localhost:8081/ping/$token`
Script path	`$script = "C:\Users\shubh\OneDrive\Documents\Sentra\scripts\demo-cron-job.ps1"`
Command to run	`$run = "conhost.exe --headless powershell.exe -NoProfile -ExecutionPolicy Bypass -File $script -Token $token"`
Create task	`schtasks /Create /TN "SentraDemoJob" /SC MINUTE /MO 1 /TR $run /F`
Allow on battery	`$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries` `Set-ScheduledTask -TaskName "SentraDemoJob" -Settings $settings`

During the demo

I want to show...	Type
The job running	`schtasks /Run /TN "SentraDemoJob"`
The job failing	`New-Item -ItemType File "$env:TEMP\sentra-demo-backups\FAIL"` then Run
Recovering from failure	`Remove-Item "$env:TEMP\sentra-demo-backups\FAIL"` then Run
The job silently stopping	`schtasks /Change /TN "SentraDemoJob" /DISABLE` and wait about 2 min
The job coming back	`schtasks /Change /TN "SentraDemoJob" /ENABLE` then Run
What the job did	`Get-Content "$env:TEMP\sentra-demo-backups\job.log" -Tail 5`
Clean up	`schtasks /Delete /TN "SentraDemoJob" /F`, then delete the check in the dashboard

Three sentences to explain it to the teacher

1. "A cron job can fail silently, so Sentra makes the job report in: it calls a secret ping URL every time it finishes."
2. "If the job reports a failure, or the ping doesn't arrive within the period plus the allowed delay, Sentra marks it DOWN and opens an incident."
3. "This is a dead man's switch, a push model: the job contacts Sentra, so Sentra never needs access to the job's machine."